

P1108R2

web_view

Published Proposal, 2019-06-16

This version:

<http://wg21.link/p1108r2>

Author:

[Hal Finkel](#) (Argonne National Laboratory)

Audience:

SG13, SG18

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes a web_view facility for the C++ standard library. This facility enables modern, natural, multimodal user interaction by leveraging existing web standards and technologies.

Table of Contents

- 1 **Change Log**
- 2 **Introduction**
- 3 **Proposed Interface**
- 4 **An Example**
- 5 **Very-Preliminary Wording**
- 6 **Acknowledgments**

References

Informative References

§ 1. Change Log

- r2: Based on the SG13 discussion in San Diego, added some comments on deployment options and process separation.
- r1: Added discussion of Qt. Added links to public discussion threads in the acknowledgments.

§ 2. Introduction

After a long and interestingly-introspective graphics evening session in Rapperswil, we did not have consensus to move forward in the 2D graphics space. Nevertheless, this is an important area (and see sections 4 and 5 of [\[P0939R0\]](#)), wide interest remains, and in this paper I propose taking a different approach from those previously discussed. In particular, I propose adding a `web_view` class to the standard library. I view this as taking the path represented by [\[P1062R0\]](#) to its logical conclusion. This represents, in my view, both the best approach, and the only practical approach, we can take to enable useful graphical user interaction in standard C++. In addition, graphics is not enough. We need to enable modern, natural, multimodal user interaction in order for a facility to be broadly useful for application development.

I believe that:

1. The underlying use case, which is unfortunately not well described as "2D Graphics", is important.
2. The underlying approach of [\[P1062R0\]](#), by making use of outside standards (i.e., SVG), embraces the right philosophical approach to improving standard C++ in this regard.

Reality is that most users do not interact with applications using a command prompt (i.e., console I/O), but rather, use some graphical user interface. The C++ standard, however, provides no useful facilities in this regard, and as a result, users either need to make use of system-specific APIs, third-party libraries, or move to a different programming language. Moreover, these interactions require both input and output facilities, where output includes 2D graphics, 3D graphics, text rendering, dialog elements, and there are many forms of input devices (especially when accessibility-related technologies are considered). While I believe that the authors of [\[P0267R7\]](#), and others, have made a commendable effort in the 2D-graphics area, down-scoping the problem to 2D graphics fails to address the underlying requirement of enabling user interaction through modern interfaces.

Unfortunately, this committee has neither the time nor the expertise to address this problem by directly creating some sufficiently-comprehensive API. Specifically, this is the problem addressed by web standards (i.e., HTML, CSS, SVG, and so on), those are in-turn built on many efforts in graphics (and many other areas), and clearly we cannot foster a comparable effort in this space in this committee. The only feasible way forward is to reach out to the large and vibrant community tackling this issue, creating portable standards in this space, and make direct use of their efforts.

Walking this path to its logical conclusion leads to an API providing a window into which a C++ program can inject web content (along with some callbacks to provide data, and possibly, handle some other events directly). An important question is whether this API surface could be small- and long-lived enough to be useful. To get some idea of what this space looks like, I recommend looking at:

- [The WebKit \(WebKitGTK+\) API](#)
- [The Internet Explorer IWebBrowser2 API](#)
- [The macOS/iOS WebKit API](#)
- [The Android WebView API](#)

There are a number of concepts exposed in these APIs, including things like sandboxing parameters, which seem likely be too dynamic to reasonably standardize. We'd be relying on the library to provide "reasonable defaults" in many cases. That having been said, as with our Unicode support in standard C++, our API surface could increase over time. To be useful, we'll need to require support for a large number external standards (i.e., [X]HTML, CSS, SVG, ECMAScript, and possibly others). Our three-year release cycle is likely sufficient to maintain a proper list of such standards, but it's still a large list, and to be clear, the transitive closure of this list is huge.

Interacting with HTML, etc. is cumbersome in C++ without additional libraries (especially as presented in the example in this paper). I fully expect that, if we decide to go down this route, additional utilities and abstractions will be added to go along with it. Also, even with only the presented interface, I can certainly imagine constructing a program with mostly-static HTML input combined with some kind of reactive JavaScript library (e.g., [Vue.js](#)) that's reasonably clean (at least in the sense that it does not involve the gratuitous composition of markup text in C++, although it might involve use of JavaScript injection to update values).

One natural question that has been asked is: Why don't we just standardize Qt's API? Qt is not the only framework in this space, but it seems representative. From <http://doc.qt.io/qt-5/classes.html>, there are 1,594 classes in the Qt 5 API. From <http://doc.qt.io/qt-5/functions.html>, there are 12,984 functions. Dealing with such a large API seems impractical for our committee process, and frankly, this still seems impractical even if we were to strip this down significantly. Specifically, I don't really see the committee taking a hands-off approach to the content of the interface. Even if we could start with the text of Qt's documentation, we'd need to make sure that each class and function were specified in

sufficient detail to allow for an independent (clean-room) implementation. I'm sure that suggestions would be made to change aspects of the interface to better expose capabilities of different vendors' underlying implementations, and future systems, and these would be discussed.

It has been further suggested that I look at QML/QtQuick, instead of QtWidgets, as a point of comparison. However, it's not clear why standardizing something which looks like QML/QtQuick would be more useful than using web content. QML also uses JavaScript. Also, the discussion of "Quick Controls 1 vs. Quick Controls 2" (<http://doc.qt.io/qt-5/qtquickcontrols2-differences.html>) seems to provide some non-trivial choices around desktop vs. mobile development. This could be a viable path for a standardization effort. It would, however, almost certainly invite design discussions around the API, the controls, etc. (because we can't hand that off to another standard, so we'd need to do that part ourselves), and we'd need a small-enough API surface for that to be practical.

§ 3. Proposed Interface

```
namespace std {
    template <typename T>
    concept URISchemeHandler = requires(T handler, const std::string &uri, st
        { handler(uri, os) } -> std::error_code;
    };

    template <typename T>
    concept CloseHandler = requires(T handler) {
        { handler() };
    };

    struct web_view {
        web_view(const std::string &title = "");

        std::future<std::error_code> display_from_uri(const std::string &uri);

        std::future<std::string> run_script(const std::string &script);

        template <typename URISchemeHandler>
        void set_uri_scheme_handler(const std::string &scheme, URISchemeHandler

        template <typename CloseHandler>
        void set_close_handler(CloseHandler handler);
    };
}
```

An implementation of this interface is available from my github page: [web_view](#). This implementation uses the wxWidgets [wxWebView](#) class which implemented in terms of the platform-native APIs listed above for IE (on Windows), WebKit (on macOS/iOS), and WebKitGTK+ (on Linux and other platforms). Qt also provides a [QtWebView](#), but cannot currently fully support the proposed interface because it does not provide URI-scheme handlers. There are several others wrappers of these kinds that I found on github embedded inside of other projects (e.g., [FireBreath](#), [TwitchSwitcher](#), and [SumatraPDF](#) (this one is IE-only, but see references therein for interesting details)).

The proposed interface does not have a function to display a web page directly from a string (or similar). Such a facility could certainly be provided given the underlying interfaces provided by the browser APIs. However, if that page is going to contain links to further content that the application will provide, then a URI-scheme handler will be needed regardless. If the content is truly static and

resides in a file, the a path to the file can be provided to the `display_from_uri` method. It should be noted that, as a limitation derived from current implementations, the URI scheme handlers provide a kind of virtual file-system interface, and as such, do not support POST data being provided to the handler along with the URI itself. To support that, and other protocols directly, we may need to provide an actual socket-based server to which the `web_view` could connect.

It is important to note that the handlers, both the URI-scheme handler, and the close handler, likely must be allowed to run in a different thread from the thread that created the `web_view` object. For security and other reasons, many implementations will separate the web-content rendering and script execution into a separate process. The `web_view` interface must allow for this, and in practice, this implies that the interface should avoid fine-grained interaction between the C++ application and the web content (as, in many implementations, all such interactions are mediated by remote procedure calls (RPC), and such calls are relatively expensive). Specifically, traversing and altering the DOM (document object model) of the content of the `web_view` from the C++ application, even if supported by the underlying platform APIs, would not only require a larger API surface, but would likely also be an expensive API to use.

Surveying the current implementations has convinced me that this kind of interface is appropriate for standardization, at least in the sense that, while broadly useful, using these services from a C++ application today requires difficult-to-get-right platform-specific code. Moving that burden to C++ library implementers, as a result, makes sense. In addition, it should be possible to create production applications using this facility that meet modern user expectations across many different kinds of devices and platforms.

I think that it's important that we think of this interface as one exposing underlying platform capabilities, not as one requiring an implementation shipped as part of the C++ standard library. Required security and functionality updates on many platforms for web-content support may be far more frequent than updates to the C++ standard library, and a `web_view` in the C++ standard library should automatically use this frequently-updated web-content software.

I don't believe that we can require all C++ implementations, not even all non-freestanding C++ implementations, to provide a web-content interface. As a result, it must be legal for an implementation to stub out the implementation of `web_view` where it cannot be reasonably supported. Nevertheless, given currently-shipping web-browser implementations, we can provide a succinct API that ties C++ into the most-vibrant standards-driven ecosystem in this space.

§ 4. An Example

Here's an example application which uses the proposed interface. It is the test in the prototype implementation repository, and while not the simplest possible application, it makes use of all of the parts of the proposed interfaces, and moreover, makes it clear that asynchronous programming may be unavoidable in this space.

```

#include <web_view>

#include <vector>
#include <string>
#include <thread>
#include <chrono>
using namespace std::chrono_literals;

int main(int argc, char *argv[]) {
    std::vector<std::string> args(argv, argv + argc);

    std::mutex m;
    std::condition_variable cv;
    bool done = false;

    std::web_view w("web_view test app");
    w.set_uri_scheme_handler("wv", [&](const std::string &uri, std::ostream &os) {
        std::cout << "request: " << uri << "\n";
        os << "<html><head><title>" << uri << "</title></head><body><p>" << uri;
        for (auto &a : args)
            os << "<tr><td>" << a << "</td></tr>" << "\n"; // we need some kind of table
        os << "</table>";
        os << "<p><a href=\"\" << uri << "/more.html\" << "\">more</a></p>";
        os << "<ul id='dl'></ul>";
        os << "</body></html>";

        return true;
    });
    w.set_close_handler([&]() {
        std::unique_lock<std::mutex> ul(m);
        done = true;
        ul.unlock();
        cv.notify_one(); });

    auto f = w.display_from_uri("wv://first.html");
    std::cout << "initial display complete: " << f.get() << "\n";

    std::unique_lock<std::mutex> ul(m);
    while (!cv.wait_for(ul, 2000ms, [&] { return done; }))) {
        auto r = w.run_script("var node = document.createElement('li');"
                               "node.appendChild(document.createTextNode('Time h"
                               "document.getElementById('dl').appendChild(node);"
                               "var d = new Date(); d.getTime());");
    }
}

```



```

    std::cout << "got from script: " << r.get() << "\n";
}

std::cout << "web_view closed\n";

return 0;
}

```

§ 5. Very-Preliminary Wording

Header `<web_view>` synopsis [**web_view.syn**]:

The header defines a class for providing a web-content-driven interface for the purpose of external user interaction.

```

namespace std {
    template <typename T>
    concept URISchemeHandler = requires(T handler, const std::string &uri,
        { handler(uri, os) } -> std::error_code;
    };

    template <typename T>
    concept CloseHandler = requires(T handler) {
        { handler() };
    };

    class web_view;
}

```

class web_view [**web_view**]

```

class web_view {
    web_view(const std::string &title = "");

    std::future<std::error_code> display_from_uri(const std::string &uri);
    std::future<std::string> run_script(const std::string &script);

    template <typename URISchemeHandler>
    void set_uri_scheme_handler(const std::string &scheme, URISchemeHandle

    template <typename CloseHandler>
    void set_close_handler(CloseHandler handler);
};

```

Each `web_view` class instance represents an independent, asynchronous web-content interface. The provided `web_view` shall support content complying with the [\[HTML5\]](#), [\[2dcontext\]](#), [\[WebGL\]](#), [\[CSS3-UI\]](#), [\[css-cascade-3\]](#), [\[css-grid-1\]](#), [\[css-scroll-snap-1\]](#), [\[css3-images\]](#), [\[css3-background\]](#), [\[css3-namespace\]](#), [\[css-writing-modes-3\]](#), [\[css-color-3\]](#), [\[css-fonts-3\]](#), [\[css3-mediaqueries\]](#), [\[css-text-3\]](#), [\[css-text-decor-3\]](#), [\[css-values-3\]](#), [\[css-writing-modes-3\]](#), [\[css-syntax-3\]](#), [\[css3-conditional\]](#), [\[css-flexbox-1\]](#), [\[selectors-3\]](#), [\[css-will-change-1\]](#), [\[css-variables-1\]](#), [\[compositing-1\]](#), [\[CSS2\]](#), [\[WOFF\]](#), [\[SVG11\]](#), [\[PNG\]](#), [\[ECMAScript\]](#), [\[ECMA-402\]](#), [\[hr-time-2\]](#), [\[DOM-Level-3-Core\]](#), [\[DOM-Level-3-Events\]](#), [\[user-timing\]](#), [\[navigation-timing\]](#), [\[resource-timing-1\]](#), [\[tracking-dnt\]](#), [\[geolocation-API\]](#), [\[WebCryptoAPI\]](#), [\[encrypted-media\]](#), [\[mediacapture-streams\]](#), [\[beacon\]](#), [\[IndexedDB\]](#), [\[page-visibility-2\]](#), [\[ElementTraversal\]](#), [\[DOM-Level-2-Style\]](#), [\[DOM-Level-2-Traversal-Range\]](#), [\[gamepad\]](#), [\[CSP2\]](#), [\[cors\]](#), [\[upgrade-insecure-requests\]](#), [\[referrer-policy\]](#), [\[rfc7034\]](#), [\[rfc7932\]](#), [\[rfc6797\]](#), [\[rfc6066\]](#), [\[rfc2397\]](#), and [\[rfc8446\]](#) standards.

[*Note:*

Implementations are encouraged to support the latest [WHATWG living standards](#), [\[wai-aria\]](#), [\[webrtc\]](#), [\[webvtt1\]](#), and otherwise maximize compatibility with other implementations (see, e.g., [Can I use...](#))

-- end note]

`web_view` member functions [**`web_view.members`**]

```

web_view(const std::string &title = "");

```

Effects: Constructs an object of the class `web_view` with the specified title.

[*Note:*

The title should be used by the implementation to associate a name with the web content's interface consistent with how that implementation generally displays the name of an interactive application. For example, on implementations that display graphical windows with title bars, the provided title, followed by a ": ", may be prepended to any title provided by the web content itself for the purpose of setting the text displayed in the title bar. The intent is that, from the user's perspective, the title sets the name of the application associated with the web content's interface.

-- end note]

```
std::future<std::error_code> display_from_uri(const std::string &uri);
```

Effects: Causes the top-level web content to be replaced with content loaded from the provided URI. The implementation shall support the URI format specified in [\[rfc3986\]](#).

Returns: A future containing the error code describing the final status of the request. It is implementation defined at what point during the content-loading process the error status is determined. If resources are unavailable to display the requested content, then an error should be set in the returned future. The implementation shall not wait indefinitely for necessary resources to become available before setting the final error status.

[*Note:*

The implementation should handle this function as a top-level navigation request. All state associated with web content previously displayed should be rendered inaccessible to the content loaded as a result of calling this function. If the previous content caused additional windows to be opened, those windows should be closed.

-- end note]

```
std::future<std::string> run_script(const std::string &script);
```

Effects: The implementation shall execute the provided string as an [\[ECMAScript\]](#) script in the context of the current web content. The implementation may define limits on the size of the provided script and the execution time of the script.

Returns: The result of the script shall be converted to a string and set in the returned future.

```
template <typename URISchemeHandler>void set_uri_scheme_handler(const st
```

Effects: Registers the provided callable object to handle the provided URI scheme. When requests are generated to URIs with the provided scheme, the function-call operator of the provided object is invoked. The handler is provided with the full URI requested and a reference to a `std::ostream` into which the requested data should be stored. If an error is encountered, an appropriate error code should be returned. Methods on the provided handler object may be simultaneously called on threads other than the thread calling this function. Such calls need not be synchronized with each other.

```
template <typename CloseHandler>void set_close_handler(CloseHandler hand
```

Effects: Registers the provided callable object which will be called by the implementation when the web content's interface becomes irrevocably unusable.

[*Note:*

On implementations where the web content is displayed in a window, this handler should be called when the user, or other system activity, causes the window to be closed.

-- end note]

§ 6. Acknowledgments

I'd like to thank JF Bastien, Botond Ballo, Jeffrey Yasskin, Mike Spertus, Bryce Lebach, Vinnie Falco, and Chandler Carruth for feedback and suggestions.

Some public discussion is available on [reddit](#). Also, see this thread on [Gecko's list](#).

§ References

§ Informative References

[2DCONTEXT]

Rik Cabanier; et al. [HTML Canvas 2D Context](https://www.w3.org/TR/2dcontext/). 19 November 2015. REC. URL: <https://www.w3.org/TR/2dcontext/>

[BEACON]

Ilya Grigorik; et al. [Beacon](https://www.w3.org/TR/beacon/). 13 April 2017. CR. URL: <https://www.w3.org/TR/beacon/>

[COMPOSITING-1]

Rik Cabanier; Nikos Andronikos. [Compositing and Blending Level 1](https://www.w3.org/TR/compositing-1/). 13 January 2015. CR. URL: <https://www.w3.org/TR/compositing-1/>

[CORS]

Anne van Kesteren. [Cross-Origin Resource Sharing](https://www.w3.org/TR/cors/). 16 January 2014. REC. URL: <https://www.w3.org/TR/cors/>

[CSP2]

Mike West; Adam Barth; Daniel Veditz. [Content Security Policy Level 2](https://www.w3.org/TR/CSP2/). 15 December 2016. REC. URL: <https://www.w3.org/TR/CSP2/>

[CSS-CASCADE-3]

Elika Etemad; Tab Atkins Jr.. [CSS Cascading and Inheritance Level 3](https://www.w3.org/TR/css-cascade-3/). 28 August 2018. CR. URL: <https://www.w3.org/TR/css-cascade-3/>

[CSS-COLOR-3]

Tantek Çelik; Chris Lilley; David Baron. [CSS Color Module Level 3](https://www.w3.org/TR/css-color-3/). 19 June 2018. REC. URL: <https://www.w3.org/TR/css-color-3/>

[CSS-FLEXBOX-1]

Tab Atkins Jr.; et al. [CSS Flexible Box Layout Module Level 1](https://www.w3.org/TR/css-flexbox-1/). 19 November 2018. CR. URL: <https://www.w3.org/TR/css-flexbox-1/>

[CSS-FONTS-3]

John Daggett; Myles Maxfield; Chris Lilley. [CSS Fonts Module Level 3](https://www.w3.org/TR/css-fonts-3/). 20 September 2018. REC. URL: <https://www.w3.org/TR/css-fonts-3/>

[CSS-GRID-1]

Tab Atkins Jr.; Elika Etemad; Rossen Atanasov. [CSS Grid Layout Module Level 1](https://www.w3.org/TR/css-grid-1/). 14 December 2017. CR. URL: <https://www.w3.org/TR/css-grid-1/>

[CSS-SCROLL-SNAP-1]

Matt Rakow; et al. [CSS Scroll Snap Module Level 1](https://www.w3.org/TR/css-scroll-snap-1/). 31 January 2019. CR. URL: <https://www.w3.org/TR/css-scroll-snap-1/>

[CSS-SYNTAX-3]

Tab Atkins Jr.; Simon Sapin. [CSS Syntax Module Level 3](#). 20 February 2014. CR. URL: <https://www.w3.org/TR/css-syntax-3/>

[CSS-TEXT-3]

Elika Etemad; Koji Ishii; Florian Rivoal. [CSS Text Module Level 3](#). 12 December 2018. WD. URL: <https://www.w3.org/TR/css-text-3/>

[CSS-TEXT-DECOR-3]

Elika Etemad; Koji Ishii. [CSS Text Decoration Module Level 3](#). 3 July 2018. CR. URL: <https://www.w3.org/TR/css-text-decor-3/>

[CSS-VALUES-3]

Tab Atkins Jr.; Elika Etemad. [CSS Values and Units Module Level 3](#). 31 January 2019. CR. URL: <https://www.w3.org/TR/css-values-3/>

[CSS-VARIABLES-1]

Tab Atkins Jr.. [CSS Custom Properties for Cascading Variables Module Level 1](#). 3 December 2015. CR. URL: <https://www.w3.org/TR/css-variables-1/>

[CSS-WILL-CHANGE-1]

Tab Atkins Jr.. [CSS Will Change Module Level 1](#). 3 December 2015. CR. URL: <https://www.w3.org/TR/css-will-change-1/>

[CSS-WRITING-MODES-3]

Elika Etemad; Koji Ishii. [CSS Writing Modes Level 3](#). 24 May 2018. CR. URL: <https://www.w3.org/TR/css-writing-modes-3/>

[CSS2]

Bert Bos; et al. [Cascading Style Sheets Level 2 Revision 1 \(CSS 2.1\) Specification](#). 7 June 2011. REC. URL: <https://www.w3.org/TR/CSS2/>

[CSS3-BACKGROUND]

Bert Bos; Elika Etemad; Brad Kemper. [CSS Backgrounds and Borders Module Level 3](#). 17 October 2017. CR. URL: <https://www.w3.org/TR/css-backgrounds-3/>

[CSS3-CONDITIONAL]

David Baron. [CSS Conditional Rules Module Level 3](#). 4 April 2013. CR. URL: <https://www.w3.org/TR/css3-conditional/>

[CSS3-IMAGES]

Elika Etemad; Tab Atkins Jr.. [CSS Image Values and Replaced Content Module Level 3](#). 17 April 2012. CR. URL: <https://www.w3.org/TR/css3-images/>

[CSS3-MEDIAQUERIES]

Florian Rivoal; et al. [Media Queries](#). 19 June 2012. REC. URL: <https://www.w3.org/TR/css3-mediaqueries/>

[CSS3-NAMESPACE]

Elika Etemad. [CSS Namespaces Module Level 3](#). 20 March 2014. REC. URL: <https://www.w3.org/TR/css-namespaces-3/>

[CSS3-UI]

Tantek Çelik; Florian Rivoal. [CSS Basic User Interface Module Level 3 \(CSS3 UI\)](#). 21 June 2018. REC. URL: <https://www.w3.org/TR/css-ui-3/>

[DOM-Level-2-Style]

Chris Wilson; Philippe Le Hégarret. [Document Object Model \(DOM\) Level 2 Style Specification](#). 13 November 2000. REC. URL: <https://www.w3.org/TR/DOM-Level-2-Style/>

[DOM-Level-2-Traversal-Range]

Joseph Kesselman; et al. [Document Object Model \(DOM\) Level 2 Traversal and Range Specification](#). 13 November 2000. REC. URL: <https://www.w3.org/TR/DOM-Level-2-Traversal-Range/>

[DOM-Level-3-Core]

Arnaud Le Hors; et al. [Document Object Model \(DOM\) Level 3 Core Specification](#). 7 April 2004. REC. URL: <https://www.w3.org/TR/DOM-Level-3-Core/>

[DOM-Level-3-Events]

Gary Kacmarcik; Travis Leithead; Doug Schepers. [UI Events](#). 8 November 2018. WD. URL: <https://www.w3.org/TR/uievents/>

[ECMA-402]

[ECMAScript Internationalization API Specification](#). URL: <https://tc39.github.io/ecma402/>

[ECMAScript]

[ECMAScript Language Specification](#). URL: <https://tc39.github.io/ecma262/>

[ElementTraversal]

Doug Schepers; Robin Berjon. [Element Traversal Specification](#). 22 December 2008. REC. URL: <https://www.w3.org/TR/ElementTraversal/>

[ENCRYPTED-MEDIA]

David Dorwin; et al. [Encrypted Media Extensions](#). 18 September 2017. REC. URL: <https://www.w3.org/TR/encrypted-media/>

[GAMEPAD]

Scott Graham; Theodore Mielczarek; Brandon Jones. [Gamepad](#). 14 December 2018. WD. URL: <https://www.w3.org/TR/gamepad/>

[geolocation-API]

Andrei Popescu. [Geolocation API Specification 2nd Edition](#). 8 November 2016. REC. URL: <https://www.w3.org/TR/geolocation-API/>

[HR-TIME-2]

Ilya Grigorik; James Simonsen; Jatinder Mann. [High Resolution Time Level 2](#). 1 March 2018. CR. URL: <https://www.w3.org/TR/hr-time-2/>

[HTML5]

Ian Hickson; et al. [HTML5](#). 27 March 2018. REC. URL: <https://www.w3.org/TR/html5/>

[IndexedDB]

Nikunj Mehta; et al. [Indexed Database API](#). 8 January 2015. REC. URL: <https://www.w3.org/TR/IndexedDB/>

[MEDIACAPTURE-STREAMS]

Daniel Burnett; et al. [Media Capture and Streams](#). 3 October 2017. CR. URL: <https://www.w3.org/TR/mediacapture-streams/>

[NAVIGATION-TIMING]

Zhiheng Wang. [Navigation Timing](#). 17 December 2012. REC. URL: <https://www.w3.org/TR/navigation-timing/>

[P0267R7]

Michael B. McLaughlin, Herb Sutter, Jason Zink, Guy Davidson. [A Proposal to Add 2D Graphics Rendering and Display to C++](#). 10 February 2018. URL: <https://wg21.link/p0267r7>

[P0939R0]

B. Dawes, H. Hinnant, B. Stroustrup, D. Vandevorode, M. Wong. [Direction for ISO C++](#). 10 February 2018. URL: <https://wg21.link/p0939r0>

[P1062R0]

Bryce Adelstein LeBlach, Olivier Giroux, Zach Laine, Corentin Jabot, Vittorio Romeo. [Diet Graphics](#). 7 May 2018. URL: <https://wg21.link/p1062r0>

[PAGE-VISIBILITY-2]

Ilya Grigorik; Arvind Jain; Jatinder Mann. [Page Visibility Level 2](#). 17 October 2017. PR. URL: <https://www.w3.org/TR/page-visibility-2/>

[PNG]

Tom Lane. [Portable Network Graphics \(PNG\) Specification \(Second Edition\)](#). 10 November 2003. REC. URL: <https://www.w3.org/TR/PNG/>

[REFERRER-POLICY]

Jochen Eisinger; Emily Stark. [Referrer Policy](#). 26 January 2017. CR. URL: <https://www.w3.org/TR/referrer-policy/>

[RESOURCE-TIMING-1]

Arvind Jain; et al. [Resource Timing Level 1](#). 30 March 2017. CR. URL: <https://www.w3.org/TR/resource-timing-1/>

[RFC2397]

L. Masinter. [The "data" URL scheme](#). August 1998. Proposed Standard. URL: <https://tools.ietf.org/html/rfc2397>

[RFC3986]

T. Berners-Lee; R. Fielding; L. Masinter. [Uniform Resource Identifier \(URI\): Generic Syntax](#). January 2005. Internet Standard. URL: <https://tools.ietf.org/html/rfc3986>

[RFC6066]

D. Eastlake 3rd. [Transport Layer Security \(TLS\) Extensions: Extension Definitions](#). January 2011. Proposed Standard. URL: <https://tools.ietf.org/html/rfc6066>

[RFC6797]

J. Hodges; C. Jackson; A. Barth. [HTTP Strict Transport Security \(HSTS\)](#). November 2012. Proposed Standard. URL: <https://tools.ietf.org/html/rfc6797>

[RFC7034]

D. Ross; T. Gondrom. [HTTP Header Field X-Frame-Options](#). October 2013. Informational. URL: <https://tools.ietf.org/html/rfc7034>

[RFC7932]

J. Alakuijala; Z. Szabadka. [Brotli Compressed Data Format](#). July 2016. Informational. URL: <https://tools.ietf.org/html/rfc7932>

[RFC8446]

E. Rescorla. [The Transport Layer Security \(TLS\) Protocol Version 1.3](#). August 2018. Proposed Standard. URL: <https://tools.ietf.org/html/rfc8446>

[SELECTORS-3]

Tantek Çelik; et al. [Selectors Level 3](#). 6 November 2018. REC. URL: <https://www.w3.org/TR/selectors-3/>

[SVG11]

Erik Dahlström; et al. [Scalable Vector Graphics \(SVG\) 1.1 \(Second Edition\)](#). 16 August 2011. REC. URL: <https://www.w3.org/TR/SVG11/>

[TRACKING-DNT]

Roy Fielding; David Singer. [Tracking Preference Expression \(DNT\)](#). 17 January 2019. NOTE. URL: <https://www.w3.org/TR/tracking-dnt/>

[UPGRADE-INSECURE-REQUESTS]

Mike West. [Upgrade Insecure Requests](#). 8 October 2015. CR. URL: <https://www.w3.org/TR/upgrade-insecure-requests/>

[USER-TIMING]

Jatinder Mann; Zhiheng Wang; Anderson Quach. [User Timing](#). 26 February 2019. REC. URL: <https://www.w3.org/TR/user-timing-1/>

[WAI-ARIA]

James Craig; Michael Cooper; et al. [Accessible Rich Internet Applications \(WAI-ARIA\) 1.0](#). 20 March 2014. REC. URL: <https://www.w3.org/TR/wai-aria/>

[WebCryptoAPI]

Mark Watson. [Web Cryptography API](#). 26 January 2017. REC. URL: <https://www.w3.org/TR/WebCryptoAPI/>

[WebGL]

Dean Jackson; Jeff Gilbert. [WebGL 2.0 Specification](#). 12 August 2017. URL: <https://www.khronos.org/registry/webgl/specs/latest/2.0/>

[WEBRTC]

Adam Bergkvist; et al. [WebRTC 1.0: Real-time Communication Between Browsers](#). 27 September 2018. CR. URL: <https://www.w3.org/TR/webrtc/>

[WEBVTT1]

Silvia Pfeiffer. [WebVTT: The Web Video Text Tracks Format](#). 10 May 2018. CR. URL: <https://www.w3.org/TR/webvtt1/>

[WOFF]

Jonathan Kew; Tal Leming; Erik van Blokland. [WOFF File Format 1.0](#). 13 December 2012. REC. URL: <https://www.w3.org/TR/WOFF/>